객체지향 프로그래밍 고급

4강 : 객체 포인터

Chapter 4

namespace

객체의 동적 메모리 할당과 객체 포인터

this 포인터

namespace

- **프로그램을 구성하는 내용들을 논리적 구조로 묶어 하나의 독립된 특정 범위로 표시하는 것**
- **특정 영역에 이름을 붙여주기 위한 문법적 요소**

```
namespace 네임스페이스명
{
    몸체
}
```

```
#include <iostream>

void SimpleFunc()
{
    std::cout << " BestCom이 정의한 함수" << std::endl;
}
void SimpleFunc()
{
    std::cout << "ProgCom이 정의한 함수" << std::endl;
}
int main()
{
   SimpleFunc();
   return 0;
}
```

X

```
#include <iostream>

namespace BestCom
{
    void SimpleFunc()
    {
            std::cout << " BestCom이 정의한 함수"
            << std::endl;
    }
}
namespace ProgCom
{
    void SimpleFunc()
    {
            std::cout << "ProgCom이 정의한 함수"
             << std::endl;
    }
}
```

```
int main()
{
   BesstCom::SimpleFunc();
   ProgCom::SimpleFunc();
   return 0;
}
```

using

```
namespace BestCom
{
  .....
}

using namespace BestCom;

using BestCom::SimpleFunc;
```

```
#include <iostream>

int main()
{
    std::cout << “Welcome.” << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main()
{
    cout << “Welcome.” << endl;
    return 0;
}
```

Chapter 4

namespace

객체의 동적 메모리 할당과 객체 포인터

this 포인터

객체 포인터

- **객체 포인터는 객체의 주소값을 저장하는 포인터 변수이다.**

클래스명 *객체명;
객체포인터명 = &객체명;
객체포인터명->멤버변수명;

```
Dog marry;
Dog *pt;
pt = &marry;
pt->ShowData();
```

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
Private:
    string name;
    int   age;
    int   weight;
Public:
    Dog(){};
    Dog(string n, int a, int w);
    void ShowData();
};

Dog::Dog(string n, int a, int w)
{
   name = n;
   age = a;
   weight = w;
}
```

```
void Dog::ShowData()
{
   cout << "나이 : " << age << endl;
   cout << "몸무게 : " << weight << endl;
}

int main()
    Dog marry("happy", 5, 6);
    Dog *pt;
    pt = &marry;
    pt->ShowData();
    marry.ShowData();
    return 0;
}
```

[예제1]

- 학생의 국어, 영어 성적을 입력받아 학생의 총점과 평균을 계산하고 출력하는 프로그램을 작성하라. 클래스의 객체는 포인터를 이용하라.

홍길동
국어 : 100
영어 : 99
총점 : 199
평균 : 99.5

```
#define SUBJECT 2
class Score
{
Private:
    string name;
    int   Kor;
    int   Eng;
    int   tot;
    double avg;
Public:
    Score(string n, int k, int e);
    void getTotal();
    void ShowData();
};

Score::Score(string n, int a, int w)
{
   name = n;
   Kor = k;
   Eng = e;
}
```

```
void Score::getTotal()
{
   tot = Kor + Eng;
   avg = (double)tot/SUBJECT;
}
void Score::ShowData()
{
     . . . .
}

int main()
    Score st("홍길동", 100, 99);
    Score *pt = &st;
    pt->getTotal();
    pt->ShowData();
    return 0;
}
```

객체의 동적 메모리 할당

- new 연산자를 사용하여 객체를 선언하면 메모리의 heap 영역에 동적으로 할당된다. 이 때 자동으로 생성자가 호출되어 초기화된다.

```
클래스명 *객체명;
객체명 = new 클래스명;
```

```
Dog  * ptr;
ptr = new Dog();
```

- new 연산자로 생성된 객체는 포인터로 처리되므로 멤버변수, 멤버함수를 사용할 때는 다음과 같이 사용한다.

```
객체명->멤버변수,   객체명->멤버함수()
```

```
ptr->name;
ptr->ShowData();
```

객체의 동적 메모리 할당

- 동적 할당 후 delete 연산자로 동적 영역을 해제해야 한다.

```
Dog *pt;
pt = new Dog(“happy”, 5, 6);
pt->ShowData();
delete pt;
```

✓ 객체를 동적할당하는 곳은? → 객체를 생성하는 함수

✓ 동적할당을 해제하는 곳은? → 객체를 생성하는 함수

멤버 변수의 동적 메모리 할당

- 멤버 변수를 동적 할당하기 위해서는 (1) class 내에서 포인터로 멤버 변수를 선언하고, (2) 생성자에서 해당 포인터 변수를 이용하여 동적 할당한다.

✓ 멤버변수를 동적할당하는 곳은? → 생성자

✓ 메모리 해제하는 곳은? → 소멸자

멤버 변수의 동적 메모리 할당

```
class Dog {
private:
    char *name;
    int age;
    int weight;
public:
    Dog() { };
    Dog(char *n, int a, int w);
    ~Dog();
    void ShowData();
};
Dog::Dog(char *n, int a, int w) {
    name = new char[20];
    strncpy(name, n, 20);
    age = a;
    weight = w;
}
```

```
Dog::~Dog() {
    delete[] name;
}
void Dog::ShowData() {
    cout << "나이: " << age << endl;
    cout << "몸무게: " << weight << endl;
}
int main() {
    Dog *pt = new Dog("happy", 5, 6);
    pt->ShowData();
    delete pt;
    return 0;
}
```

[예제2]

X, Y 좌표값을 갖는 Point 클래스를 설계하고, 동적메모리를 할당하여 객체를 생성하고(x, y)가 (3, 10)인 좌표를 출력하시오.

[예제3]

이름, 주소, 나이를 관리하는 Person 클래스를 설계하고 자신의 정보를 출력하시오.

이 때 이름과 주소 변수는 각각 char *로 선언하고 동적 할당하라.

또한 메인함수에서 객체 생성 시 동적 메모리 할당을 사용하라.

[예제4]

정수값으로 반지름을 입력받고 면적을 출력하는 Circle 클래스를 구현하라.

Circle 객체는 동적 생성해야 한다.

Chapter 4

namespace

객체의 동적 메모리 할당과 객체 포인터

this 포인터

this 포인터

- this 포인터는 객체에 대한 포인터로 멤버 함수를 소유한 각 객체를 가리킨다.
- 즉, 객체에 대한 주소이다.
- 클래스 내부에서는 기본적으로 this 포인터를 사용한다.
- 클래스에서 일반 멤버함수는 this 포인터를 기술하지 않아도 컴파일러가 자동으로 this를 붙이지만 표시되지는 않는다.

this 포인터의 활용

```
class SoSimple
{
private:
    int num;
public:
    void ThisFunc(int num);
    {
            this->num = 207;
            num = 10;
    }
};
```

this 포인터로 이름 충돌 해결하기

```
class Dog
{
private:
	char name[20];
	unsigned int age;
	unsigned int weight;
public:
	Dog() { };
	Dog(char name[], int age, int weight);
	void showData();
};
```

```
Dog::Dog(char name[], int age, int weight)
{
	strncpy(this->name, name, 20);
	this->age = age;
	this->weight = weight;
}
```

this 포인터를 이용한 자기 자신 확인

```
using namespace std;

class WhoAmI
{
public:
	int id;
	WhoAmI(int id_arg);
	void ShowYourself() const;
};
WhoAmI::WhoAmI(int id_arg)
{
		id = id_arg;
}
void WhoAmI::ShowYourself() const
{
		cout << id << " this: " << this << endl;
}
```

```
int main()
{
	WhoAmI obj1(1);
	WhoAmI obj2(2);
	WhoAmI obj3(3);

	obj1.ShowYourself();
	obj2.ShowYourself();
	obj3.ShowYourself();

	cout << &obj1 << endl;
	cout << &obj2 << endl;
	cout << &obj3 << endl;

   return 0;
}
```

this 포인터

```
class Simple
{
private:
   int num;
public:
        Simple(int n);
        Void ShowData();
        Simple *getThisPointer() {return this;}
};
Simple::Simple(int n)
{
        num = n;
        cout << "address : " << this << endl;
}
```

```
void Simple::ShowData()
{
                cout << num << endl;
}


int main()
{
        Simple obj(100);
        Simple *ptr1 = obj.GetThisPointer();
        cout << ptr1 << endl;
        ptr1->ShowData();
        return 0;
}
```

[예제4] this 포인터 사용

이름(name), 나이(age), 몸무게(weight)를 멤버변수로 갖는 Dog 클래스를 정의하고 키보드로부터 이름, 나이, 몸무게를 입력받고, 출력하라.
(this 포인터를 이용하라)

Self-Reference 반환

```
class SoSimple
{
private:
   int num;
public:
   SoSimple(int n) : num(n)
   {
   }
   SoSimple& Adder(int n)
   {
            num += n;
            return *this;
   }
      void ShowData()
      {
            cout << num << endl;
      }
};
```

```
int main()
{
      SoSimple obj(3);
      SoSimple &ref = obj.Adder(2);

      obj.ShowData();
      ref.ShowData();

      return 0;
}
```

[심화]

```
class SoSimple
{
private:
   int num;
public:
   SoSimple(int n) : num(n) {}
   SoSimple& Adder(int n)
   {
            num += n;
            return *this;
   }
      SoSimple& ShowData()
      {
            cout << num << endl;
            return *this;
      }
};
```

```
int main()
{
      SoSimple obj(3);
      SoSimple &ref = obj.Adder(2);

      obj.ShowData();
      ref.ShowData();

      ref.Adder(1).ShowData().Adder(5).ShowData();
      return 0;
}
```

[예제] *this 포인터로 연속 사용

아래는 이름(name), 나이(age), 몸무게(weight)를 멤버변수로 갖는 Dog 클래스이다.
키보드로부터 이름, 나이, 몸무게를 입력받고, 이 값을 저장하는 생성자를 this 포인터를 사용하여 정의하시오.
멤버변수값(이름, 나이, 몸무게)을 매개변수로 받아 저장한 후 *this 포인터를 반환하는 멤버함수 SetData를 정의하시오.

```
char name[20];
unsigned int age;
unsigned int weight;
```

```
class Dog
{
private:
	string name;
	unsigned int age;
	unsigned int weight;
public:
	Dog() { };
	Dog(string name, int age, int weight);
	Dog& setData(string name, int age, int weight);
	void showData();
};
Dog::Dog(string name, int age, int weight)
{
	this->name = name;
	this->age = age;
	this->weight = weight;
}
void Dog::showData()
{
	cout << name << " " << age << " " << " " << weight << endl;
}
```

```
Dog& Dog::setData(string name, int age, int weight)
{
	this->name = name;
	this->age = age;
	this->weight = weight;
	return *this;
}

int main()
{
	Dog d("happy", 3, 4);
	d.showData();
	d.setData("nurry", 5, 6).showData();

return 0;
}
```

[예제5] Circle과 CircleManager

다음은 이름과 반지름을 속성으로 가진 Circle 클래스와 이들을 배열로 관리하는 CircleManager 클래스이다. 원의 개수를 입력하고, 각각의 이름과 반지름을 입력한 후, 모든 원의 면적을 출력하라. 이 중 10보다 큰 면적을 갖는 원의 이름을 출력하라.

```
class Circle {
private:
   int radius;
   string name;  // 원의 이름
public:
   Circle();
   void setCircle(string name, int radius);
   double getArea();
   string getName();
};
```

```
class CircleManager {
private:
   Circle *p;
   int size;
public:
   CircleManager(int size);
   ~CircleManager();
   void searchByArea();
};
```

[예제6]

다음 클래스에서 Person은 사람을, Family는 가족을 추상화한 클래스이다. 프로그램이 실행되도록 클래스를 구현하라.

```
class Person
{
private:
   string name;
public:
   Person(string name) { this->name =
name; }
   string getName();
};
```

```
class Family
{
private:
   Person *p;
   int size;  // 가족 구성원 수
public:
   Family(int size);
   void show();
   ~Family();
};
```

```
int main()
{
   Family *simpson = new Family( 3);
   simpson->setName(0, “Mr.Simpson”);
   simpson->setName(1, “Mrs.Simpson”);
   simpson->setName(2, “Bart Simpson”);
   simpson->show();
   delete simpson;
}
```